Grado en Ingeniería Robótica en Sistemas de Información

Robótica y Visión Artificial
ROBOTICS CHALLENGE UPO 24/25

Autores: Juan Rodríguez López y Luis Carmona Berdugo

Métodos implementados

Constructor

Inicialmente, se obtienen los parámetros establecidos del challenge y se crean los objetos y variables adicionales para controlar los distintos parámetros del robot con unos valores medidos y testeados en simulaciones para comprobar su eficacia.

```
class TurtlebotController():

    def __init__(self, rate):

        robot_vel_topic = rospy.get_param('~robot_vel_topic', 'speed')
        robot_scan_topic = rospy.get_param('~robot_scan_topic', 'laser')

        # read the maximum linear and angular velocities
        # from the parameter server!!!!
        self.max_lin_vel = rospy.get_param('~max_lin_vel', 0.2)  # m/s
        self.max_ang_vel = rospy.get_param('~max_ang_vel', 0.4)  # rad/s

        self.r_soi = 1 # Sphere of influence radius

        self.rate = rate # Hz  (1/Hz = secs)
        # we store the received path here
        self.path = Path()
        self.path_received = False
        self.laser = LaserScan()
        self.laser_received = False
        self.is_collision = False

        # Declare the velocity command publisher
        self.cmd_vel = rospy.Publisher(robot_vel_topic, Twist, queue_size=10)

        self.listener = tf.TransformListener()

        self.utils = Utils(self.listener, "odom")

        # subscription to the scan topic [sensor_msgs/LaserScan]
        rospy.Subscriber(robot_scan_topic, LaserScan, self.laserCallback)

        # subscription to the path topic [nav_msgs/Path]
        rospy.Subscriber("path", Path, self.pathCallback)
```

Path planner

Algoritmo A*

En este proyecto, se ha hecho uso del algoritmo A* para obtener la planificación del camino que sigue el robot en el transcurso de la simulación.

```python
class Planner:
    def plan(self, sx, sy, gx, gy):
        # TODO: fill the rest of the function
        print("Start node:", start_node)
        print("Goal node:", goal_node)
        start_node.g = 0
        start_node.f = self.heuristic(start_cell_x, start_cell_y, goal_cell_x, goal_cell_y)
        visited = []
        no_visited = [start_node]
        while len(no_visited) > 0:
            current = no_visited[0]
            for node in no_visited:
                if node.f < current.f or (node.f == current.f and node.h < current.h):
                    current = node

            directions = [[-1, -1],[0, -1],[1, -1],[-1, 0],[1, 0],[-1, 1],[0, 1],[1, 1]]
            no_visited.remove(current)
            visited.append(current)

            # If (current == goal) return Success
            if current.x_cell == goal_node.x_cell and current.y_cell == goal_node.y_cell:
                break
            for d in directions:
                dx = current.x_cell + d[0]
                dy = current.y_cell + d[1]
                node = self.get_node(dx, dy)
                if node is None:
                    node = self.update_node(dx, dy, math.inf, goal_node, None)
                if not self.node_is_valid(node):
                    continue
                cost = np.linalg.norm(np.array(d))
                if node.g > current.g + cost:
                    g = current.g + cost
                    node = self.update_node(node.x_cell, node.y_cell, g, goal_node, current)
                    if node not in no_visited and node not in visited:
                        no_visited.append(node)
        self.nodes = {}
        if goal_node.parent is None:
            return None
        x, y = self.cell2real(goal_node.x_cell, goal_node.y_cell)
        rx = [x]
        ry = [y]
        current = goal_node.parent
        while current is not None:
            x, y = self.cell2real(current.x_cell, current.y_cell)
            rx.append(x)
            ry.append(y)
            current = current.parent
        rx.reverse()
        ry.reverse()
        return rx, ry
```

Como se observa en el código, elegimos el mejor siguiente nodo a visitar en función del resultado obtenido en base su heurística y distancia acumulada.

Heurística

Adicionalmente, como métrica establecida para calcular la heurística de cada nodo, hemos elegido la distancia de Manhattan.

```python
# Manhattan distance
def heuristic(self, x, y, goal_x, goal_y):
    return abs(goal_x - x) + abs(goal_y - y)
```

Operaciones “get_node” y “update_node”

```python
def get_node(self, x, y):
    hash_id = f"{int(x)};{int(y)}"
    if hash_id in self.nodes:
        return self.nodes[hash_id]
    else:
        return None


def update_node(self, x, y, g, goal, parent):
    hash_id = f"{int(x)};{int(y)}"
    if hash_id in self.nodes:
        node = self.nodes[hash_id]
        node.g = g
        node.f = node.g + node.h
        node.parent = parent
        return node
    else:
        node = self.Node(x, y, g, self.h(x, y, goal), parent)
        self.nodes[hash_id] = node
        return node
```

Como métodos adicionales para la organización de los nodos creados en este código, se desarrollan los métodos “get_node” y “update_node” para obtener un nodo dado en base a sus coordenadas, y actualizar sus datos en el caso en que sea necesario, respectivamente.

Robot Controller

Método “Command”

En este código se realizan las órdenes estrictas y necesarias para guiar al robot hasta el objetivo final, controlando así sus velocidades, posición actual y técnicas de detección de colisiones potenciales y evasión de obstáculos.

El método escogido para ducha evasión de obstáculos en base a las normas de este challenge es “Potential Fields”, limitándose así las velocidades angular y lineal.

```python
class TurtlebotController():
    def command(self):

        # TODO: use current_goal
        # Put your control law here (copy from EPD1)
        angular = 0.0
        linear = 0.0
        min_d = 1.0
        angle_range_difference = False

        # Normalize directionection towards goal
        current_goal = np.array([current_goal.pose.position.x, current_goal.pose.position.y])
        current_distance = np.linalg.norm(current_goal)
        current_goal_normalized = current_goal / current_distance

        # Calculate linear and angular velocities

        target = np.array([current_goal_normalized[0], current_goal_normalized[1]])
        angle = math.atan2(current_goal_normalized[1], current_goal_normalized[0])

        allowed_angle = 15
        allowed_angle = allowed_angle * math.pi / 180

        angular = (angle / abs(angle)) * self.max_ang_vel
        if abs(angle) <= allowed_angle * 1.25:
            angular = angular * 0.25

        speed_factor = 1
        if min_d < self.r_soi:
            speed_factor = (min_d / self.r_soi) * 0.5 + 0.5
        if angle_range_difference:
            speed_factor = 0.33
        if abs(angle) <= allowed_angle:
            linear = self.max_lin_vel * speed_factor
            linear = min(linear, self.max_lin_vel)

        # check the maximum speed values allowed

        linear, angular = self.checkMaximumSpeedValues(linear,angular,self.max_lin_vel,self.max_ang_vel)

        # If the computed commands does not provoke a collision,
        # send the commands to the robot
        # TODO: fill the checkCollision function
        if(self.checkCollision(linear, angular)==False):
            self.publish(linear,angular)
            return False

        # if a possible collision is detected,
        # try to find an alternative command to
        # avoid the collision
        # TODO: fill the CollisionAvoidance function
        linear, angular = self.collisionAvoidance()

        # check the maximum speed values allowed
        linear, angular = self.checkMaximumSpeedValues(linear,angular,self.max_lin_vel,self.max_ang_vel)

        self.publish(linear,angular)
        return False
```

Método “checkMaximumSpeedValues”

Para controlar que las velocidades no superen los límites establecidos debido a la ganancia que se les suma en las operaciones de guiado y evasión del robot, se hace uso del método “checkMaximumSpeedValues”.

```
def checkMaximumSpeedValues(self,linear,angular,max_lin_vel,max_ang_vel):

    if(abs(angular) > max_ang_vel):
        if(angular < 0):
            angular = -max_ang_vel
        else:
            angular = max_ang_vel
    if(linear > max_lin_vel):
        linear = max_lin_vel

    return linear,angular
```

Método “goalReached”

Esta función “goalReached” verifica si un robot ha alcanzado su objetivo final. Utiliza la posición final de una ruta para calcular la distancia actual al objetivo, devolviendo True si está lo suficientemente cerca (menos de 0.15 metros) y False en caso contrario.

```
def goalReached(self):
    # ------------------------------------------------------------
    # TODO: use the last point of the path to check if the robot
    # has reached the final goal (the robot is in a close position).
    # return True if the FINAL goal was reached, False otherwise
    # ------------------------------------------------------------
    if len(self.path.poses) == 0:
        return False

    # Get goal position
    final_goal = self.path.poses[-1]
    final_goal_rf = self.utils.transformPose(final_goal, 'base_footprint')

    # Calculate current_distance
    final_goal = np.array([final_goal_rf.pose.position.x, final_goal_rf.pose.position.y])
    current_distance = np.linalg.norm(final_goal)

    if current_distance > 0 and current_distance < 0.15:
        return True

    return False
```

Método “getSubGoal”

La función “getSubGoal” selecciona un subgoal en la ruta del robot. Encuentra el punto más cercano al robot y, usando un desplazamiento (lookahead), devuelve una posición futura transformada como próximo subgoal.

```python
def getSubGoal(self):
    # ------------------------------------------------------------
    # TODO: use self.path.poses to find the subgoal to be reach
    # You could transform the path points to the robot reference
    # to find the closest point:
    # path_pose = self.path.poses[index]
    # path_pose_rf = self.utils.transformPose(path_pose, 'base_footprint')

    # Return False in case that path has not loaded
    if len(self.path.poses) == 0:
        return False

    # Define lookahead in the path positions
    lookahead = 2

    # Calculate index of point with lowest current_distance to the robot
    current_distance = math.inf
    index = -1
    for i, point in enumerate(self.path.poses):
        # Calculate current_distance for point
        point_rf = self.utils.transformPose(point, 'base_footprint')
        point_rf = np.array([point_rf.pose.position.x, point_rf.pose.position.y])
        distance_to_goal = np.linalg.norm(point_rf)

        if distance_to_goal < current_distance:
            index = i
            current_distance = distance_to_goal

    # Get index and point according to the lowest current_distance + lookahead
    index = min(index + lookahead, len(self.path.poses) - 1)
    subgoal_pose = self.path.poses[index]
    subgoal = self.utils.transformPose(subgoal_pose, 'base_footprint')
    # ------------------------------------------------------------
    return subgoal
```

Método “checkCollision”

La función “checkCollision” detecta posibles colisiones usando datos del sensor láser. Evalúa la distancia a obstáculos y devuelve True si están demasiado cerca, o False si es seguro avanzar.

```python
def checkCollision(self, linear, angular):
    # ------------------------------------------------------------
    # TODO: use self.laser to check possible collisions
    # Optionally, you can also use the velocity commands
    # return True if possible collision, False otherwise

    # When laser data has not loaded, return False
    if not self.laser:
        rospy.loginfo("No information laser received")
        return False
    if len(self.laser.ranges) == 0:
        rospy.loginfo("No information laser received")
        return False

    if np.min(self.laser.ranges) > self.r_soi + 0.12:
        self.is_collision = False
        return False

    if self.is_collision or np.min(self.laser.ranges) < self.r_soi:
        self.is_collision = True
        return True


    return False
    # ------------------------------------------------------------
```

Método “collisionAvoidance”

La función “collisionAvoidance” calcula velocidades para evitar colisiones mediante campos potenciales. Combina una fuerza atractiva hacia el “subgoal” y una fuerza repulsiva del obstáculo más cercano. Ajusta la dirección y velocidad del robot según la cercanía del obstáculo y la desviación angular. Retorna velocidades lineal y angular para maniobrar de forma segura y reactivar el avance.

```python
def collisionAvoidance(self):
    # ------------------------------------------------------------
    # TODO: try to find an alternative command to avoid the collision
    # Here you must try to implement one of the reactive methods
    # seen in T4: bug algorithm, potential fields, velocity obstacles,
    # Dynamic Window Approach, others...
    # Feel free to add the new variables and methods that you may need

    index = np.argmin(self.laser.ranges)
    angle = (self.laser.angle_min + index * self.laser.angle_increment)
    min_d = min(np.min(self.laser.ranges), 1)

    # Calculate nearest obstacle
    x = self.laser.ranges[index] * math.cos(angle)
    y = self.laser.ranges[index] * math.sin(angle)

    repulsive_force = np.array([-x, -y])

    current_goal_pose = self.getSubGoal()
    goal_direction = np.array([current_goal_pose.pose.position.x, current_goal_pose.pose.position.y])
    goal_direction_normalized = (goal_direction / np.linalg.norm(goal_direction))

    goal_force = goal_direction_normalized
    c = 0
    if min_d <= self.r_soi:
        c = (self.r_soi - min_d) / min_d
    repulsive_force_normalized = repulsive_force / np.linalg.norm(repulsive_force)
    potential_field_force = (repulsive_force_normalized) * c

    goal_w = 4
    potential_field_w = 1.25
    target_force = goal_force * goal_w + potential_field_force * potential_field_w

    allowed_angle = 45
    allowed_angle = allowed_angle * math.pi / 180
    target_force_normalized = target_force / np.linalg.norm(target_force)
    angle_range_difference = abs(np.arccos(np.clip(np.dot(goal_direction_normalized, target_force_normalized), -1, 1))) > allowed_angle

    angular = 0.0
    linear = 0.0

    target = np.array([target_force[0], target_force[1]])
    angle = math.atan2(target_force[1], target_force[0])

    allowed_angle = 15
    allowed_angle = allowed_angle * math.pi / 180

    angular = (angle / abs(angle)) * self.max_ang_vel
    if abs(angle) <= allowed_angle * 1.25:
        angular = angular * 0.25

    speed_factor = 1
    if min_d < self.r_soi:
        speed_factor = (min_d / self.r_soi) * 0.5 + 0.5
    if angle_range_difference:
        speed_factor = 0.33
    if abs(angle) <= allowed_angle:
        linear = self.max_lin_vel * speed_factor
        linear = min(linear, self.max_lin_vel)

    lin_vel = linear
    ang_vel = angular

    # ------------------------------------------------------------
    return lin_vel, ang_vel
```